Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Proposal
on
"Vaani:
An Offline AI Personal Assistant"

[Code No.: COMP 311]

(For partial fulfillment of III Year/ I Semester in Computer Science)

Submitted by:

Suprabha Bhuju (2)

Avipsa Parajuli (37)

Aayusha Shrestha (50)

Pratistha Thapa (57)

Submitted to:

Mr. Suman Shrestha

Department of Computer Science and Engineering

Submission Date: 14/11/2025

Abstract

The proposed project, *Vaani: An Offline AI Personal Assistant*, is a voice-driven software system designed to function entirely without internet connectivity. It interprets spoken commands, processes them locally, and executes corresponding actions through a fully offline pipeline. The system integrates three key components: Vosk for on-device speech recognition, pyttsx3 for speech synthesis, and a Python-based logic engine that handles intent analysis and task execution. By performing all operations on the local device, Vaani ensures user privacy, consistent performance in low-connectivity environments, and independence from cloud services. Combining speech processing, natural language understanding, and automation, this project demonstrates a self-sufficient AI assistant capable of operating reliably in offline conditions.

Keywords: local processing, natural language understanding, system automation, self-sufficient digital assistant

Table of Contents

List of Figures

Acronyms/Abbreviations

AI	Artificial Intelligence
LLM	Large Language Models
STT	Speech-to-Text
TTS	Text-to-Speech
GUI	Graphical User Interface
AMD	Advanced Micro Devices
CUDA	Compute Unified Device Architecture

Chapter 1 Introduction

In today's rapidly advancing digital landscape, artificial intelligence has become deeply embedded in everyday life through applications like voice assistants and smart devices. However, most existing assistants such as Siri, Google Assistant, and Alexa require constant internet connectivity, making them inaccessible in areas with poor network coverage or limited data availability. This dependency on cloud-based systems also raises privacy concerns, as users' voice data is transmitted and stored on remote servers.

To address these limitations, our project proposes the development of an Offline AI-Powered Personal Assistant, a lightweight, privacy-first digital companion capable of functioning entirely without internet access. The assistant will process voice commands locally, perform everyday tasks such as setting reminders, opening applications, telling the time, and responding to general queries using a local knowledge base. By combining offline speech recognition, natural language understanding, and text-to-speech technologies, this system will provide the convenience of an AI assistant while maintaining data security and accessibility in low-connectivity environments.

This project is especially relevant in regions like Nepal, where many communities experience unreliable internet connectivity. Students, professionals, and rural residents often require quick digital assistance for simple tasks but cannot depend on cloud-based tools. The offline assistant provides an inclusive solution that empowers users with reliable, secure, and efficient personal computing support regardless of network conditions. Beyond accessibility, it also demonstrates the potential of edge AI, performing intelligent computation directly on local devices aligning with global trends toward decentralized and privacy-preserving technology.

1.1 Background

Voice assistants have revolutionized human-computer interaction by making technology more natural and conversational. However, their reliance on remote servers limits their

reach and usability. Current solutions like Google Assistant and Alexa depend on large-scale cloud infrastructure, which introduces two major challenges:

1. Connectivity dependency - Users require a stable internet connection for even simple commands.

2. Privacy concerns - Voice data and personal interactions are often stored on external servers.

Our proposed Offline AI-Powered Personal Assistant bridges this gap by integrating open-source speech and AI technologies to run directly on a user's local device. Using lightweight speech-to-text engines such as Vosk or Whisper (small models), and offline text-to-speech systems like Pyttsx3, the assistant interprets commands and performs tasks without sending data to external servers. This project builds on the emerging field of on-device intelligence, a growing movement in AI research and development that focuses on reducing cloud dependence and enhancing user privacy. It highlights the potential for deploying advanced machine learning capabilities on personal hardware, even with limited computational power.

1.2 Objectives

- Develop an Offline Personal Assistant: Build a fully functional digital assistant that operates without internet connectivity.

- Enable Core Functionalities: Implement commands such as opening applications, setting reminders, and retrieving time/date information.

- Ensure Data Privacy: Keep all speech and text processing on the local device, with no cloud storage or data sharing.

- Design Modular Architecture: Create a scalable system that can later integrate AI-based intent recognition and local knowledge bases.

- Optimize for Low-Resource Devices: Ensure smooth performance on standard laptops and mobile devices without requiring high-end hardware.

- Enhance Accessibility: Provide digital assistance to communities and users in low-connectivity or remote regions.

1.3 Motivation and Significance

The project is motivated by two major challenges in current AI systems, network dependency and data privacy. Many users, especially in developing regions, do not have access to stable internet but still require assistance for daily computing tasks. Moreover, the transmission of personal data to third-party servers poses a serious privacy risk, especially for sensitive interactions.

By developing an offline personal assistant, we aim to democratize access to AI technology enabling people to benefit from digital automation without depending on global cloud infrastructure. The system promotes digital inclusivity, allowing users in low-resource environments to experience the convenience of AI locally.

In addition, this project serves as an educational initiative, helping students explore the integration of speech processing, natural language understanding, and system automation. It demonstrates how complex AI applications can be achieved through efficient algorithms and modular design, even without massive computational resources.

1.4 Expected Outcomes

- **Functional Offline Assistant:** A working system capable of recognizing voice commands and performing key tasks offline.
- **Improved Privacy:** Complete on-device data handling, ensuring user voice and text data remain secure.
- **Accessibility for All Users:** A tool usable in both connected and disconnected environments.

- **Scalable Framework:** A modular foundation that can be expanded with AI-based Q&A or local database search.
- **User Empowerment:** Increased self-sufficiency and convenience for students, professionals, and rural residents.
- **Educational Value:** Enhanced understanding of speech processing, modular system design, and offline AI integration.

Chapter 2 Related/Existing Systems

2.1 Layla AI

Layla AI claims to be the world's first private AI. It is a chatbot that runs in offline mode on a device. It does not require internet connection and filters. It also provides complete privacy. It can be used as a personal assistant, friend, roleplay partner and many more. It also offers mini-apps, local text-to-speech and image generation. Some additional features of Layla's technology that make it unique are its advanced machine learning, natural language processing, contextual understanding and on-device processing. Despite all these features, it is not a free app which limits its accessibility.

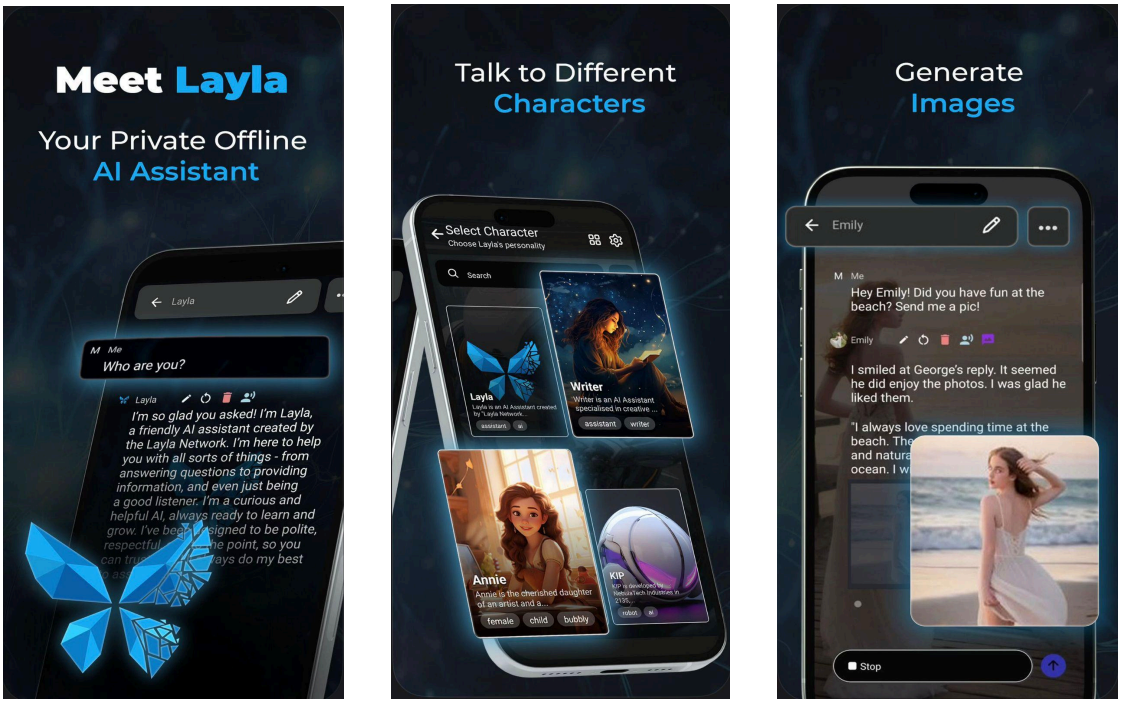

Fig 2.1.1: Interface of Layla AI

2.2 Local AI Assistant

Local AI Assistant is an advanced, offline chatbot. It is designed to allow AI-powered conversations and assistance directly to the desktop without requiring any internet connection. It uses the Microsoft Phi-3 language model, which is optimized for fast,

high-quality responses locally. It provides advanced features such as context retention, multilingual support, on-device knowledge base loading, and task-specific assistance for summarization, brainstorming, or document analysis. It offers a simple, user-friendly interface and requires no account or login. All interactions and data processing are conducted locally, ensuring your data remains private and is never transmitted outside the device.

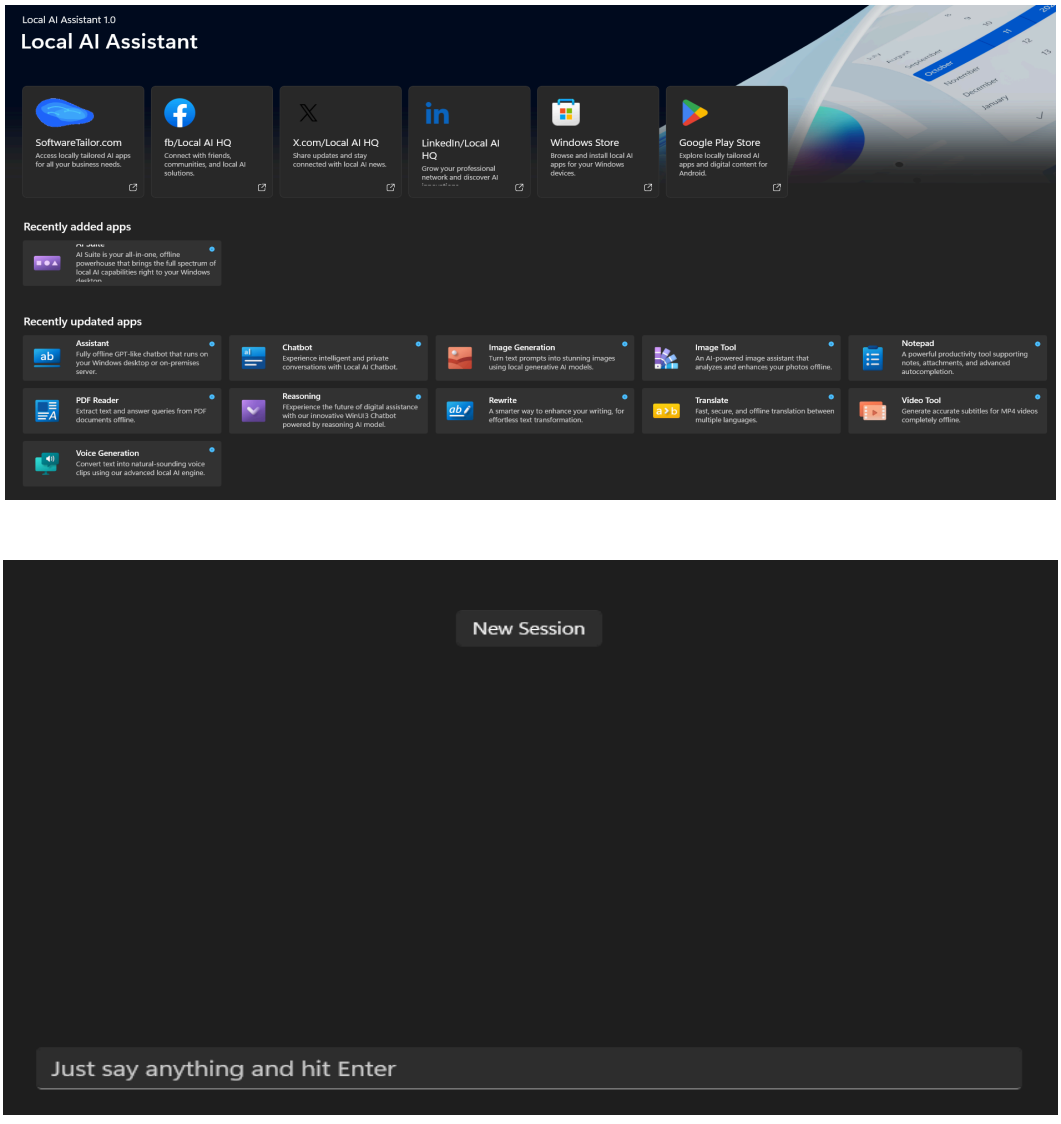

Fig 2.2.1: Interface of Local AI Assistant

2.3 Local AI - Offline AI chatbot

Local AI - Offline AI Chatbot, is an Android-based application designed to provide a completely offline conversational experience. The app enables users to interact with an AI assistant without any internet connection, ensuring privacy and data security by keeping all processing on the device itself. Its lightweight model allows smooth, real-time responses and supports natural language understanding for general queries and tasks. The application serves as a strong reference point for developing offline intelligent systems, particularly in its focus on privacy, efficiency, and accessibility.

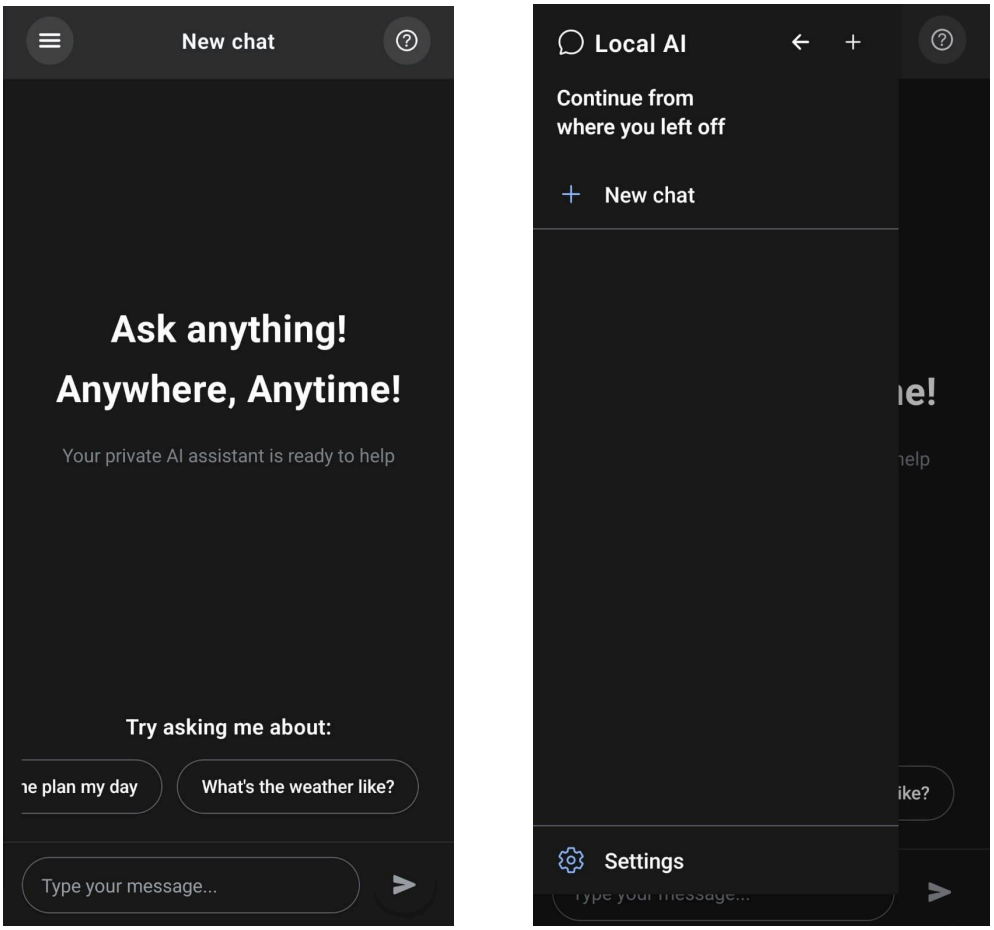

Fig 2.3.1: Interface of Local AI - Offline AI Chatbot

2.4 Enclave AI

Enclave AI is an offline, privacy-focused AI assistant for iOS and macOS that processes all data locally, ensuring that user interactions remain fully private. It supports multiple open-source models, allows users to upload documents for question-answering, and enables custom assistant personalities and workflows.It advertises features such as voice chat, custom AI assistants, document-based Q&A, and integration with Siri, Shortcuts, and iMessage, while using multiple open-source models for flexibility. Its key strengths are strong privacy, offline usability, and flexibility in model choice and personalization.

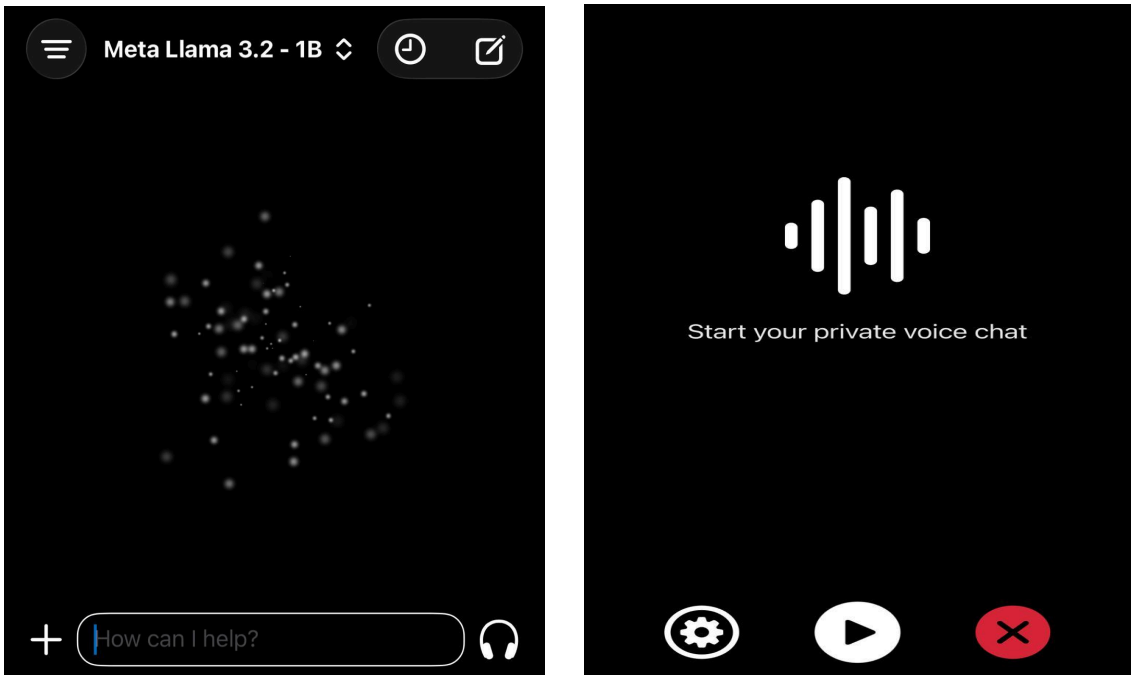

Fig 2.4.1: Interface of Enclave AI

2.5 Secret AI: Offline AI Chat

Secret AI: Offline AI Chat is an Android app that delivers a fully offline AI assistant experience, running entirely on-device without requiring internet access, accounts, or cloud services. It supports multiple advanced open-source models and formats, allowing users to perform multi-turn conversations, text generation, and even local image analysis while keeping all data private. Its strengths include complete on-device

privacy, flexibility in model selection, and offline usability, making it suitable for privacy-sensitive or low-connectivity environments.

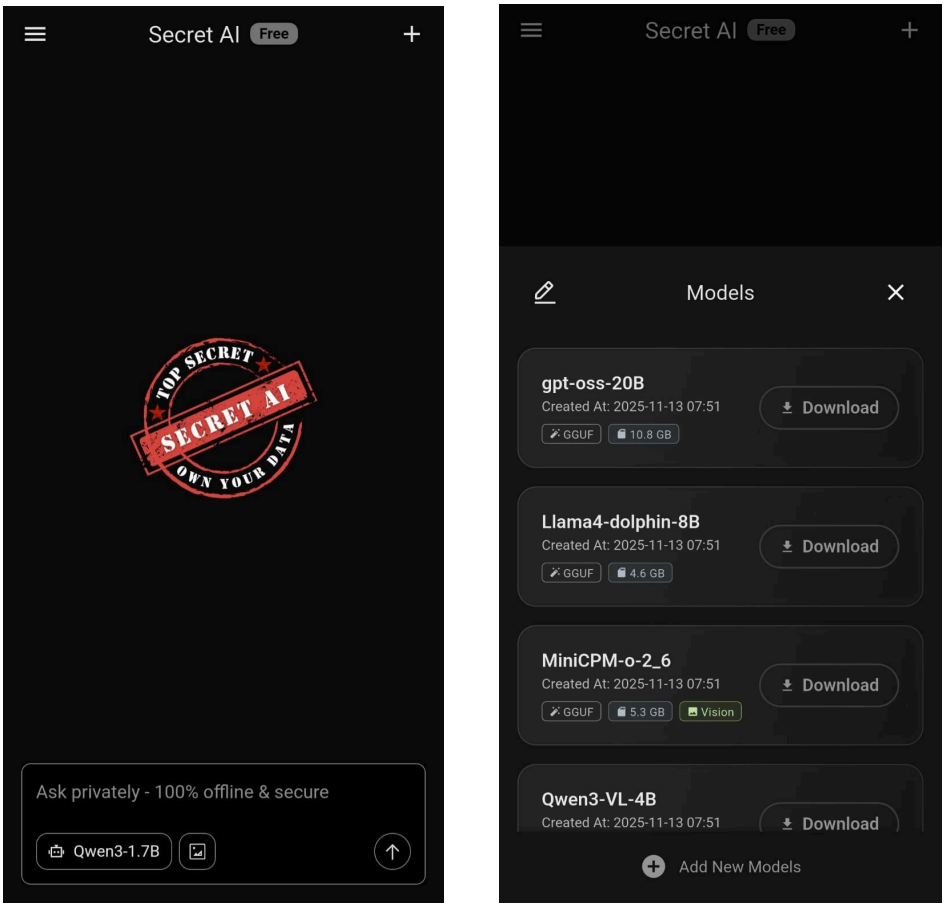

Fig 2.5.1: Interface of Secret AI

Chapter 3 Procedures and Methods

The Offline AI-Powered Personal Assistant follows a systematic design approach to develop an intelligent, voice-driven system capable of operating entirely without Internet connectivity. The project integrates core technologies such as Vosk for speech-to-text processing, pyttsx3 for text-to-speech output, and a Python-based backend for natural language understanding and command execution. Its architecture ensures fully local processing, prioritizing privacy, accessibility, and real-time response even in low-resource environments.

3.1 Requirement Analysis

The initial phase focuses on identifying functional and non-functional requirements through observation and scenario-based analysis. Informal discussions with users highlight the limitations of conventional cloud-based assistants, such as Siri or Alexa, which fail to operate in low-connectivity or offline settings. Key requirements include accurate offline speech recognition, smooth voice responses, task automation (e.g., reminders, app control), and a lightweight local knowledge base. Comparative evaluation of existing systems informs the need for an assistant that functions independently while maintaining essential AI capabilities.

3.2 System Design

The architecture of the Offline AI Assistant emphasizes local autonomy and modular design for flexibility and scalability. The system consists of the following main components:

- **Speech Processing Module:** Utilizes Vosk for offline speech-to-text recognition and pyttsx3 for natural speech output, enabling full duplex voice interaction without cloud dependency.
- **Brain Module (Command Parser & Logic Core):** Built in Python, it interprets text commands through rule-based parsing and, optionally, lightweight local models to generate contextual responses.

- **Action Execution Layer:** Handles system-level operations such as launching local applications, setting reminders, reading the date or time, and fetching data from a pre-loaded offline knowledge base.
- **Data Management Layer:** Stores reminders, responses, and user preferences locally in structured files , ensuring complete data privacy and persistence.

This design ensures the assistant performs efficiently across different hardware platforms while maintaining a modular structure for future upgrades.

3.3 Development

Development proceeds iteratively, focusing on building and integrating independent modules step by step:

- **Phase 1:** Establish the speech interface by implementing Vosk for offline voice recognition and pyttsx3 for speech synthesis.
- **Phase 2:** Develop the rule-based natural language parser and link it with the assistant's action handler.
- **Phase 3:** Integrate a local data storage system for reminders and basic Q&A retrieval.
- **Phase 4:** Add optional LLM integration for natural conversation and expanded command understanding.

Development follows a modular coding approach using Python packages and virtual environments to manage dependencies, ensuring cross-platform compatibility and maintainability

3.4 Testing

Testing is conducted to validate performance, accuracy, and reliability in offline operation:

- **Unit Testing:** Each module (STT, TTS, parser, and actions) is tested independently to ensure proper functionality.
- **Integration Testing:** Modules are combined and tested for seamless workflow from speech input to action execution.

- **Performance Testing:** The assistant's response time, CPU usage, and memory efficiency are evaluated on low-spec systems.
- **User Testing:** End-users interact with the assistant to assess accuracy, usability, and naturalness of voice interaction. Feedback informs refinements to command recognition and response handling.

3.5 Deployment

The final application is designed for cross-platform offline deployment, supporting Windows, Linux, and mobile systems. It can be distributed as a standalone Python executable generated using PyInstaller for desktop environments, and includes local installation instructions along with configuration scripts to simplify the setup process for users. Since the assistant operates completely offline, all its functionalities remain accessible without requiring Internet connectivity or cloud-based accounts.

3.6 Future Improvements

The Offline AI Assistant is designed for extensibility and long-term adaptability. Future enhancements include:

- Integration of custom voice profiles and multilingual support.
- Expansion of the offline knowledge base using local Wikipedia datasets.
- Implementation of emotion aware responses and personalized dialogue memory.
- Development of a mobile version with offline speech models for Android devices.
- Optimization through lightweight neural models for faster inference and reduced storage footprint.

These improvements aim to make the assistant more interactive, context-aware, and accessible, providing a robust foundation for future advancements in offline artificial intelligence.

Chapter 4 System Requirements

4.1 Software Specifications

4.1.1 Frontend Tools

The front-end of the Offline AI-Powered Personal Assistant is implemented through a **Command Line Interface (CLI)** built entirely in **Python**. This interface allows users to interact with the system using both voice and text commands, ensuring simplicity and platform independence.

Audio input is captured through the **speech-recognition** or microphone module, enabling the assistant to process voice commands directly from the device's built-in or external microphone. The system integrates **Whisper** or **Vosk** for offline speech-to-text conversion, allowing the assistant to accurately interpret user speech without relying on internet connectivity.

All recognized text and system responses are displayed directly in the terminal window, maintaining a lightweight and efficient user interface. For speech output, the **pyttsx3** library is employed to convert text responses into natural-sounding speech, ensuring an interactive and hands-free experience. Together, these tools create a responsive command-line environment capable of providing full voice-based interaction while remaining completely offline.

4.1.2 Backend Tools

The backend of the system is also developed in **Python**, serving as the processing core that manages speech interpretation, command execution, and system responses. The assistant utilizes **Whisper (base.en)** or **Vosk** as offline **Speech-to-Text (STT)** engines to convert user speech into text commands.

For **Natural Language Understanding (NLU)**, a rule-based command parser interprets the intent behind user inputs, such as setting reminders, opening local applications, or retrieving system information. In later stages, optional integration with lightweight local language models like **llama.cpp** will enable richer conversational capabilities and offline Q&A functionalities using small, locally stored datasets.

To produce speech output, the backend reuses **pyttsx3**, ensuring seamless integration between text generation and voice response. The system maintains user data, reminders, and custom notes using local **JSON files or SQLite databases**, preserving privacy by avoiding external data transmission.

Additionally, the backend employs Python's built-in os, subprocess, and datetime modules to execute local actions and automate system-level tasks. Since all operations are performed locally, the assistant functions entirely offline, without requiring any form of network connectivity or remote server interaction.

4.2 Hardware Specifications

The Offline AI-Powered Personal Assistant is designed to run efficiently on standard personal computers and laptops without the need for specialized hardware.

For basic operation, the system requires at least a **dual-core processor** (such as Intel i3 or AMD equivalent), **4 GB of RAM**, and **2-4 GB of available storage** to accommodate model files and data storage. A **built-in or external microphone** is essential for capturing voice input, while standard **speakers or headphones** are needed for audio output.

For smoother performance especially when using larger local models or handling more complex tasks a **quad-core processor**, **8 GB of RAM**, and **10 GB of free storage** are recommended. A **CUDA-enabled GPU** may be utilized to accelerate model inference, though it is optional.

This hardware configuration ensures the assistant remains lightweight, functional, and fully operable in offline environments, providing reliable AI-powered assistance even in areas with limited or no internet connectivity.

Chapter 5 Project Planning

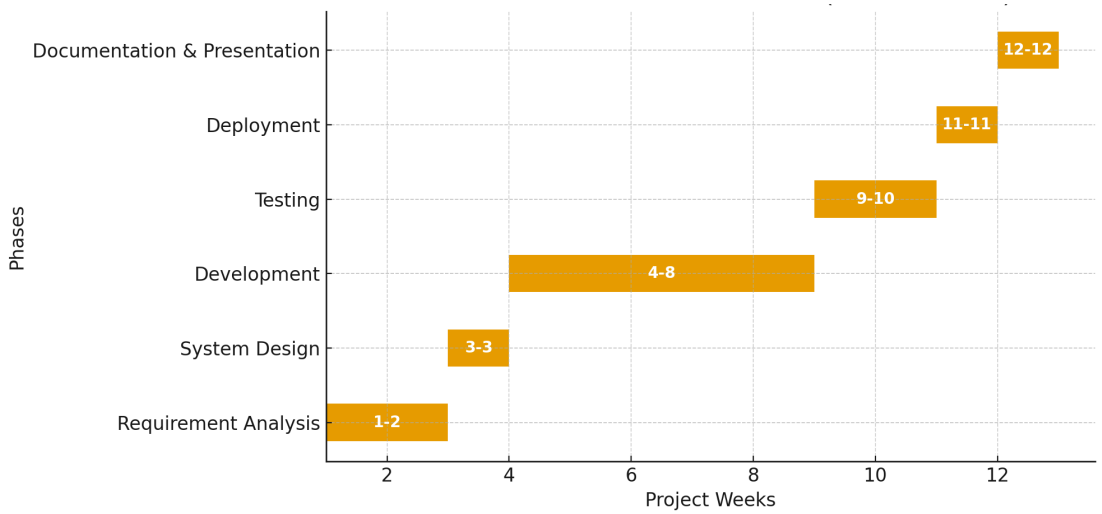

Fig 5.1.1: Gantt Chart

The development of the Offline AI Assistant follows a structured and systematic plan divided into six major phases over a period of twelve weeks. Each phase focuses on building, testing, and refining specific modules to ensure efficient progress and high-quality output. The planning approach emphasizes modular design, iterative development, and continuous testing to achieve a functional, privacy-preserving assistant capable of operating entirely offline.

5.1 Project Phases and Timeline

1. Requirement Analysis (Week 1-2):

Objective: Identify and define the project's functional and non-functional requirements through research and observation.

Tasks:

- Review existing offline AI assistants such as *Layla* and *Enclave AI* to analyze their capabilities and limitations.
- Conduct literature review and user observation to gather requirements.
- Define the system's objectives, scope, and performance benchmarks based on findings.

2. System Design (Week 3):

Objective: Develop the structural and functional design of the system, detailing component interactions and data flow.

Tasks:

- Prepare architecture diagrams outlining key modules.
- Design module interactions among the Speech Processing, Logic Core, Action Execution, and Data Management layers.
- Specify communication flow, data handling, and control logic.

3. Development (Week 4-8):

Objective: Implement and integrate the system's primary functionalities for offline AI operation.

Tasks:

- Develop offline speech recognition using Vosk.
- Integrate Text-to-Speech functionality with pyttsx3.
- Build the Rule-Based Logic Core and Command Parser.
- Implement Reminder Management and Local Data Storage using JSON or SQLite.
- Test basic integration to ensure module compatibility.

4. Testing (Week 9-10):

Objective: Evaluate the system's stability, accuracy, and offline efficiency through rigorous testing.

Tasks:

- Conduct unit, integration, and performance testing.
- Measure accuracy and response time for voice recognition and logic execution.
- Collect user feedback for usability improvements and refinements.

5. Deployment (Week 11):

Objective: Package the final version of the assistant for offline use and verify

cross-system functionality.

Tasks:

- Compile the system into a standalone executable using PyInstaller.
- Perform compatibility and reliability checks across multiple devices.
- Ensure consistent performance in offline environments.

6. Documentation and Presentation (Week 12):

Objective: Finalize and present the completed project with comprehensive technical documentation.

Tasks:

- Prepare detailed project documentation and final report.
- Create a presentation and demonstration video showcasing the system's features and operation.
- Submit all deliverables for final evaluation.

References

- Layla Network. (n.d.). *Layla - Private AI Assistant*. Retrieved November 13, 2025, from https://www.layla-network.ai/
- Local AI Assistant. (n.d.). *Local AI Assistant* [Computer software]. Microsoft Store. Retrieved November 13, 2025, from https://apps.microsoft.com/detail/9n9l7mz02t2n?hl=en-US&gl=US
- Vohra, I. (2025, August 11). *Local AI - Offline AI Chatbot* [Mobile application]. GooglePlay. Retrieved November 13, 2025, from https://play.google.com/store/apps/details?id=com.ishanvohra2.localai&hl=en
- Enclave AI. (n.d.). *Enclave AI - Offline, privacy-focused assistant*. Retrieved November 13, 2025, from https://enclaveai.app/
- Secret AI. (n.d.). *Secret AI: Offline AI Chat* [Mobile app]. Google Play Store. Retrieved November 13, 2025, from https://play.google.com/store/apps/details?id=io.secretai.llm&hl=en